Disaster Recovery Plan (DRP)

Dashboard Logistik & Rantai Pasok — Kelompok 6

26 Oktober 2026 — Versi 1.1

1. Ringkasan 2
2. Sistem yang Dilindungi 3
   2.1 Arsitektur 3
   2.2 Komponen 3
   2.3 Data 4
   2.4 Fungsi Aplikasi 4
3. Skenario Bencana 4
4. Target RTO dan RPO 5
5. Strategi Backup 5
6. Prosedur Pemulihan (Runbook) 5
   6.1 Perintah bantu 5
   6.2 D1 — Container database mati 6
   6.3 D2 — Volume hilang atau rusak 6
   6.4 D3 — Kesalahan manusia (data terhapus) 6
   6.5 D4 — Service backup berhenti 7
7. Peran dan Tanggung Jawab 7
8. Rencana DR Test 7
   8.1 Prosedur 7
   8.2 Kriteria Keberhasilan 8
   8.3 Template Hasil 8
9. Keterbatasan dan Risiko 8
10. Riwayat Revisi 9

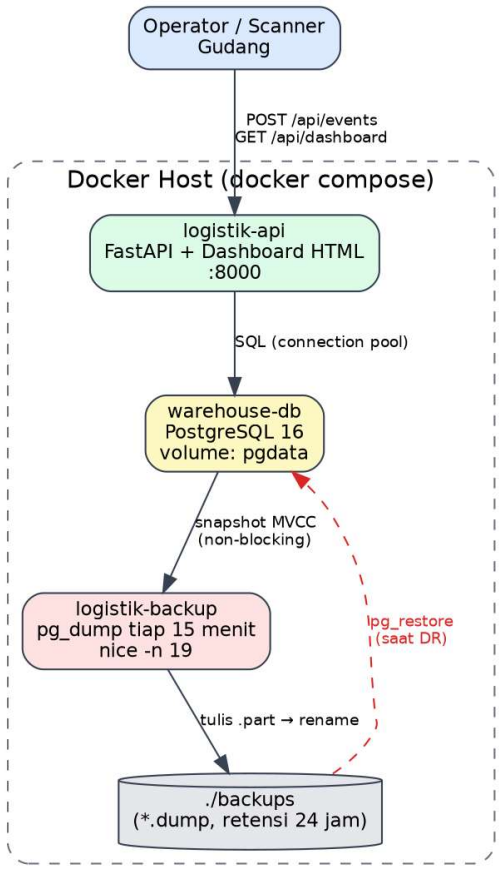
Studi kasus	Dashboard Logistik & Rantai Pasok (pemrosesan tracking barang real-time tanpa henti)
Repository	https://github.com/ammararifin298-uinsa/Cloud-Computing
Anggota	Ammar Arifin (09040624081)
	Azzam Wildan Shidqii Al-afghani (09040623054)
	Hassan Abdirahman Saman (09060624104)
	Moch. Rafli (0902062038)
	Muhammad Yusuf Multazam (09020624053)

1. Ringkasan

Item	Isi
Tujuan	Memulihkan layanan tracking logistik setelah database gudang utama gagal, dengan kehilangan data minimal
Target RTO	≤ 15 menit
Target RPO	≤ 15 menit
Strategi backup	`pg_dump` otomatis setiap 15 menit oleh container terpisah, disimpan di host (`./backups`), retensi 24 jam
Pemulihan	Restore dari file `.dump` terbaru dengan `pg_restore` (runbook pada bagian 6)
Status pengujian	**DR Test belum dijalankan.** RTO dan RPO di atas masih target, belum hasil pengukuran

2. Sistem yang Dilindungi

2.1 Arsitektur



Arsitektur sistem dan alur backup/restore

2.2 Komponen

Container	Image	Fungsi	Data / Port	Healthcheck
`warehouse-db`	postgres:16-alpine	Database gudang utama	Named volume `pgdata`; port tidak diekspos ke host	`pg_isready`
`logistik-api`	build `./api` (python:3.12-slim)	Backend API dan dashboard web	Port 8000; membaca `./backups` (read-only)	Memanggil `/health`
`logistik-backup`	postgres:16-alpine	Backup berkala `pg_dump`	Bind mount `./backups`	Ada file `.dump` berumur < 30 menit

Semua container memakai `restart: unless-stopped`. API dan backup baru dijalankan setelah database berstatus *healthy*.

2.3 Data

Tabel	Isi
`shipments`	Master pengiriman dan status terkini (kode, status, lokasi, waktu update)
`tracking_events`	Log pergerakan barang (append-only), terhubung ke `shipments` lewat `shipment_code`

2.4 Fungsi Aplikasi

Halaman	Fungsi
`/`	Ringkasan dashboard
`/checkpoint`	Mencatat checkpoint (event tracking)
`/pengiriman`	Master pengiriman
`/lacak`	Melacak riwayat satu pengiriman
`/backup`	Status backup dan prosedur DR
`/health`	Status layanan dan database (200 sehat, 503 database tidak terjangkau)

Endpoint API utama: `POST /api/events`, `GET /api/shipments`, `GET /api/shipments/{code}/events`, `DELETE /api/shipments/{code}`, `GET /api/dashboard`, `GET /api/backups`.

3. Skenario Bencana

Kode	Skenario	Kemungkinan*	Dampak*	Respons
D1	Container database mati/crash paksa (skenario DR Test)	Sedang	Tinggi	Jalankan ulang; restore bila data rusak
D2	Volume data rusak atau hilang	Rendah	Sangat tinggi	Buat ulang volume, restore dari backup
D3	Kesalahan manusia, termasuk penghapusan pengiriman lewat `DELETE /api/shipments/{code}` (menghapus riwayat event sekaligus)	Sedang	Tinggi	Restore ke database sementara, salin data yang hilang
D4	Service backup berhenti tanpa disadari	Sedang	Sedang–Tinggi	Pantau healthcheck dan halaman `/backup`, jalankan ulang

*Penilaian kualitatif oleh kelompok, belum berdasarkan data insiden nyata.

Di luar cakupan versi ini: kegagalan seluruh host/mesin (backup masih berada di host yang sama).

4. Target RTO dan RPO

Metrik	Target	Arti
RTO	**≤ 15 menit**	Waktu maksimal sistem tidak dapat menerima input tracking baru
RPO	**≤ 15 menit**	Data maksimal yang boleh hilang, sama dengan interval backup

Dasar penetapan: RPO mengikuti interval backup (900 detik); RTO ditetapkan 15 menit untuk memberi ruang bagi langkah restore manual pada data berukuran kecil. Pada kondisi terburuk (gagal tepat sebelum backup berikutnya) kehilangan data mendekati 15 menit. **Angka ini akan divalidasi dan direvisi setelah DR Test.**

5. Strategi Backup

Aspek	Keputusan
Metode	`pg_dump -Fc -Z 6` (format custom, terkompresi), snapshot konsisten tanpa mengunci tabel
Frekuensi	Tiap 900 detik (`BACKUP_INTERVAL`); backup pertama dibuat saat service backup start
Dampak performa	Container terpisah dan `nice -n 19` agar tidak membebani server utama
Penamaan	`logistik_YYYYMMDD_HHMMSS.dump`
Penyimpanan	Folder `./backups` di host, di luar volume `pgdata`
Keamanan penulisan	Ditulis ke `.part`, di-rename ke `.dump` setelah sukses; jika gagal, `.part` dihapus
Retensi	24 jam (± 96 file), file lebih lama dihapus otomatis
Pemantauan	Healthcheck container backup, halaman `/backup`, log `docker compose logs backup`

Backup pertama setelah sistem pertama kali dijalankan berisi database kosong. Pastikan ada backup yang dibuat setelah data terisi sebelum menjalankan DR Test.

6. Prosedur Pemulihan (Runbook)

Perintah ditulis untuk PowerShell di folder proyek.

6.1 Perintah bantu

```
# File backup terbaru
Get-ChildItem backups\*.dump | Sort-Object LastWriteTime -Descending |
Select-Object -First 1
```

```
# Status layanan
curl.exe http://localhost:8000/health
docker compose ps

# Jumlah event
docker compose exec db psql -U logistik -c "SELECT count(*) FROM
tracking_events;"

# Uji input baru
Invoke-RestMethod -Method Post -Uri http://localhost:8000/api/events -
ContentType "application/json" -Body '{"shipment_code":"SHP-
001","status":"DITERIMA","location":"Gudang A"}'
```

6.2 D1 — Container database mati

1. Deteksi: `/health` mengembalikan 503 atau `docker compose ps` menunjukkan `warehouse-db` tidak berjalan. **Mulai timer** (`$t0 = Get-Date`).
2. Periksa apakah container sudah hidup kembali otomatis (`docker compose ps`). Jika belum: `docker compose up -d db`.
3. Selama volume `pgdata` utuh, data kembali tanpa restore. Verifikasi dengan `/health` dan jumlah event.
4. Jika data hilang atau rusak, lanjut ke D2 langkah 4 dan seterusnya.
5. Kirim event baru. Jika berhasil, hentikan timer: `(Get-Date) - $t0`.

Catatan: dengan `restart: unless-stopped`, perilaku container setelah `docker kill` perlu diamati langsung saat uji, lalu dicatat sebagai temuan.

6.3 D2 — Volume hilang atau rusak

1. Mulai timer: `$t0 = Get-Date`
2. Hentikan dan hapus volume database: `docker compose down -v` (folder `./backups` tidak terhapus).
3. Jalankan database: `docker compose up -d db` (skema dibuat ulang oleh `init.sql`).
4. Salin backup ke container dan restore:

```
$f = (Get-ChildItem backups\*.dump | Sort-Object LastWriteTime -Descending |
Select-Object -First 1).Name
docker cp "backups\$f" warehouse-db:/tmp/restore.dump
docker compose exec db pg_restore -U logistik -d logistik --clean --if-exists
/tmp/restore.dump
```

5. Jalankan semua layanan: `docker compose up -d`
6. Verifikasi `/health`, jumlah event, lalu kirim event baru dan hentikan timer.

6.4 D3 — Kesalahan manusia (data terhapus)

1. Hentikan penulisan: `docker compose stop api`
2. Buat database sementara: `docker compose exec db createdb -U logistik pemulihan`

3. Restore backup yang dipilih ke database sementara:

```
docker cp "backups\$f" warehouse-db:/tmp/restore.dump
docker compose exec db pg_restore -U logistik -d pemulihan /tmp/restore.dump
```

4. Salin baris yang hilang dari `pemulihan` ke `logistik` (misalnya dengan `\copy`), lalu verifikasi jumlah data.
5. Hapus database sementara dan jalankan API: `docker compose start api`

6.5 D4 — Service backup berhenti

1. Periksa: `docker compose ps` dan `docker compose logs --tail 20 backup`.
2. Jalankan ulang: `docker compose up -d backup`.
3. Pastikan file `.dump` baru muncul dalam 15 menit (cek folder `backups` atau halaman `/backup`).

7. Peran dan Tanggung Jawab

Anggota	Peran
Moch. Rafli	Koordinator DR, pelaksana restore
Muhammad Yusuf Multazam	Backend/API, verifikasi data pascarestore
Ammar Arifin	Dokumentasi DRP, pencatatan dan perhitungan RTO/RPO
Hassan Abdirahman Saman	Database dan service backup
Azzam Wildan Shidqii Al-afghani	Frontend, uji input pascapemulihan

Pembagian peran adalah usulan dan dapat disesuaikan kelompok.

8. Rencana DR Test

8.1 Prosedur

Langkah	Aksi
1	Jalankan sistem, isi data uji, tunggu atau paksa backup baru (`docker compose restart backup`)
2	Catat jumlah event dan waktu event terakhir
3	Catat T0 (`$t0 = Get-Date`), lalu `docker kill warehouse-db`
4	Dokumentasikan kegagalan (`/health` 503, dashboard error)
5	Pulihkan sesuai runbook D1; untuk menguji restore dari backup, gunakan D2
6	Kirim event baru; catat T1 saat berhasil
7	RTO = T1 − T0; RPO = selisih event terakhir sebelum gangguan dengan event terakhir pada backup yang dipulihkan

8.2 Kriteria Keberhasilan

Kriteria	Ambang
RTO aktual	≤ 15 menit
RPO aktual	≤ 15 menit
Integritas	Jumlah event pascarestore sesuai isi backup yang dipakai
Fungsi	`/health` 200, dashboard tampil, event baru tersimpan

8.3 Template Hasil

Item	Hasil
Tanggal dan jam uji	
Skenario (D1/D2)	
File backup yang dipakai	
Jumlah event sebelum gangguan	
Jumlah event setelah restore	
T0 (kill)	
T1 (event baru berhasil)	
RTO aktual	
RPO aktual	
Container hidup otomatis?	
Kendala dan perbaikan	

9. Keterbatasan dan Risiko

Keterbatasan	Pengembangan
DR Test belum dijalankan; RTO/RPO belum terukur	Jalankan DR Test dan revisi dokumen
Tanpa autentikasi; `DELETE /api/shipments/{code}` dapat menghapus data tanpa otorisasi	API key/JWT, konfirmasi atau soft-delete
Password database tertulis di `docker-compose.yml`	Pindahkan ke `.env` atau Docker secrets
Backup masih satu host	Salinan offsite (cloud/object storage)
Backup belum diverifikasi isinya (hanya log dan umur file)	Verifikasi dengan `pg_restore --list` atau restore uji terjadwal
RPO dibatasi interval 15 menit	WAL archiving dan Point-in-Time Recovery
Tanpa high availability	Replikasi dan failover
Kemampuan menangani beban tracking belum diuji	Uji beban sederhana

10. Riwayat Revisi

Versi	Tanggal	Perubahan
1.0	19 Oktober 2026	DRP awal, target RTO/RPO, runbook, rencana uji
1.1	26 Oktober 2026	Nama lengkap anggota; disesuaikan dengan repository (5 halaman, endpoint baru, container backup); healthcheck 3 layanan dan restart policy; runbook PowerShell dengan `docker cp`; skenario D3 mencakup penghapusan pengiriman; daftar keterbatasan diperluas